BÖLÜM 6 / MODULE 6

Senaryolar ve Alternatif Akışlar Yazma Sanatı

Writing Scenarios and Alternative Flows

Bir senaryo adımının anatomisi: Özne-Yüklem-Nesne, topun kimde olduğu kuralı, 'validates' vs 'checks whether' tercihi, Cockburn'ün 3 turlu alternatif akış beyin fırtınası, numaralı adıma bağlanan tetikleyiciler, açık uçlu akış yasağı ve kapalı döngü (closed-loop) ritmi.

The anatomy of a step: Subject-Verb-Object, who has the ball, 'validates' vs 'checks whether', Cockburn's 3-pass alternative flow brainstorming, anchored triggers, banned open-ended flows, and the closed-loop specification rhythm.

31

BÖLÜM: SENARYOLAR VE ALTERNATIF AKIŞLAR YAZMA SANATI | MODULE: WRITING SCENARIOS AND ALTERNATIVE FLOWS

Bir Adımın Anatomisi: Özne-Yüklem-Nesne, Kuşbakışı Bakış & 'Top Kimde?' Kuralı

The Shape of a Step: Subject-Verb-Object & 'Who Has the Ball'

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
┌──────────────────────────────────────────────────────────────┐
│              BİR ADIMIN ANATOMİSİ VE 'TOP KİMDE?' KURALI     │
└──────────────────────────────────────────────────────────────┘

❌ HATALI ADIM (ÖZNESİZ / EDİLGEN / İÇERİDEN BAKIŞ):
"Tutar bakiyeden düşülür." (Edilgen!)
"Düş bakiyeden 100 TL."   (Öznesiz!)
💥 AI Alarmı: Top kimde belli değil! UI mu yapacak, servis mi yapacak, banka mı?

✅ DOĞRU ADIM (ETKEN ÇATI / TEK ÖZNE / DIŞARIDAN KUŞBAKIŞI BAKIŞ):
[ ÖZNE (Topun Sahibi) ] + [ EYLEM / FİİL ] + [ NESNE ] + [ TÜMLEÇ ]
"Sistem"               + "düşer"          + "tutarı"  + "hesap bakiyesinden."
"Üye"                  + "seçer"          + "kitabı"  + "katalog listesinden."

🎯 KURAL: Her adımın İLK KELİMESİ o an hareket yapan aktörün adı olmalıdır!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir basketbol maçını anlatan spikeri dinleyin: Spiker asla 'Top potaya girdi ve sayı oldu' deyip susmaz! 'Curry topu sürdü, pası LeBron'a attı, LeBron smacı bastı!' der. Her saniye topun kimin elinde olduğu bellidir. Eğer use case adımında 'Para çekildi' dersen spiker susmuş gibi olur; parayı hırsız mı çekti, veznedar mı verdi, ATM mi saydı bilemezsin! İlk kelime daima topu elinde tutan oyuncunun adı olmalıdır!

🇬🇧 ENGLISH (ELI5)

Listen to a live sports commentator: they never mutter passively 'The ball was hurled toward the hoop.' They say: 'Curry drives the lane, kicks the ball to LeBron, and LeBron slams it down!' At every fraction of a second, you know who holds the ball. If you write 'Money is debited', the ball-holder is invisible. Did a pickpocket steal it, or did the ledger process it? The first word of every step must name the actor holding the ball.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Bir Adımın Dilbilgisel ve Mimari Anatomisi: Cockburn use case yazımını 'temelde bir düzyazı denemesi egzersizi' olarak tanımlar. Ancak bu edebi bir üslup değil, katı bir mühendislik kuralıdır: 1) Basit Gramer: Özne, Yüklem, Nesne ve Edat öbeği. Şimdiki zaman ve etken çatı (Active Voice). 2) 'Top Kimde?' (Who Has the Ball): Her adımda tam olarak tek bir aktör hareket eder ve o aktör cümlenin ilk kelimesidir. Edilgen cümleler ('Para hesaba aktarılır') topun sahibini gizler. Oysa yapay zekâ için topun sahibi hayati önem taşır; çünkü o adımın bir Controller/UI kodu mu, bir Domain Servis kodu mu yoksa harici bir RestClient çağrısı mı olacağına bu özne karar verir! 3) Kuşbakışı Bakış (Bird's Eye View): Senaryo sistemin içinden dışarıya bakarak ('Kartı al, PIN doğrula, parayı say') yazılmaz; tüm etkileşimi yukarıdan izleyen tarafsız bir gözlemci gibi yazılır. İçeriden yazılan adımlarda kullanıcı kaybolur ve AI kullanıcı arayüzü üretmeyi unutur.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Linguistic and Architectural Shape of a Step: 1) Grammatical Strictness: One sentence per step. Present tense, active voice. Subject - Verb - Direct Object - Prepositional Phrase ('System deducts amount from account balance'). Omitted subjects ('Deduct amount') force LLMs to infer the actor, frequently assigning server logic to client components. 2) 'Who Has the Ball': Exactly one actor acts per step, declared as the very first word. Passive voice ('The loan is recorded') conceals the ball-holder. The ball-holder is the primary signal an AI uses to partition responsibilities across the architecture: actor actions become UI events; system actions become domain service methods; secondary actor actions become outbound API adapters. 3) Bird's Eye Perspective: Frame interactions from above the entire ecosystem rather than from inside the server looking out ('Read card, dispense cash' loses the human). Retaining the actor guarantees that frontend views and forms are synthesized accurately.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: 'Top Kimde' Kuralının Temiz Mimari Katmanlarına Dağılımı
package com.maang.aiup.stepshape;

public class BallHolderArchitecture {
    // 1. ÖZNE: Üye (UI / Presentation Layer) -> Top Üyede!
    public record MemberUiAction(String memberId, String bookId) {
        public void userClicksBorrowButton() {
            System.out.println("Üye butona tıkladı (Top Üyede): " + bookId);
        }
    }

    // 2. ÖZNE: Sistem (Domain Service Layer) -> Top Sistemde!
    public static class SystemDomainService {
        public void systemDeductsInventory(String bookId) {
            System.out.println("Sistem envanteri 1 azalttı (Top Sistemde): " + bookId);
        }
    }

    void main() {
        var ui = new MemberUiAction("M1", "BK-42");
        var service = new SystemDomainService();

        ui.userClicksBorrowButton();
        service.systemDeductsInventory("BK-42");
        System.out.println("✅ Adımların sorumluluğu katmanlara başarıyla yansıdı.");
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Bir adıma birden fazla aktörün hareketini sıkıştırmayın: 'Kullanıcı formu doldurur ve sistem bilgileri onaylar' YAZMAYIN! Bu iki ayrı adımdır. İlk adımda top kullanıcıda, ikinci adımda sistemdedir.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never combine two actor actions into a single compound step (e.g. 'User submits form and system validates data'). Split them. Step 1: User submits form. Step 2: System validates data.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Her adım geniş zamanlı, etken çatılı ve tek bir cümledir; ilk kelime topun sahibidir.
- Edilgen cümleler ('Para aktarılır') aktörü gizler ve AI'ın kod katmanını şaşırmasına yol açar.
- Etkileşimi sistemin içinden değil, yukarıdan kuşbakışı izleyen tarafsız bir gözle yazın.

🎯 Key Architectural Takeaways

- A step is one active-voice sentence in present tense; the first word is the ball-holder.
- Passive voice obscures responsibilities, causing AI to misallocate logic across layers.
- Write from a bird's-eye view above the interaction, preserving the user's presence.

32

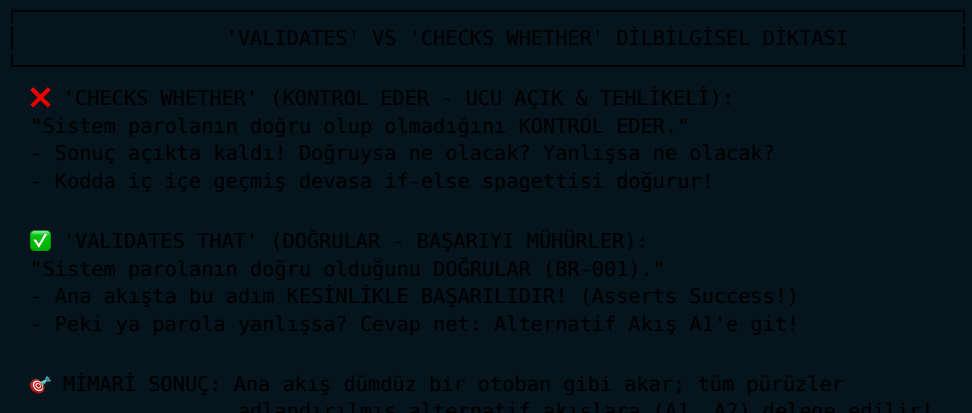
BÖLÜM: SENARYOLAR VE ALTERNATIF AKIŞLAR YAZMA SANATI | MODULE: WRITING SCENARIOS AND ALTERNATIVE FLOWS

'Validates' vs 'Checks Whether': Ana Akışta Başarıyı Mühürleme Sanatı

The 'Validates' vs 'Checks Whether' Dictum: Asserting Success in Main Flow

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Havalimanında pasaport polisine gittiğinde spiker şöyle demez: 'Polis pasaportun sahte olup olmadığına bakıyor ve düşünüyor.' Hayır! Ana senaryoda 'Polis pasaportun geçerli olduğunu doğrular' denir ve yürür geçersin. Eğer pasaport sahteyse, o zaman polis düğmeye basar ve seni yan odaya (Alternatif Akış A1) çekerler! 'Doğrular' kelimesi ana yolda yürümeyi sağlar; soru işaretlerini çöpe atar!

🇬🇧 ENGLISH (ELI5)

At passport control, the script does not read: 'Officer ponders whether the visa is forged.' The main scenario asserts: 'Border officer validates that the visa is authentic.' You proceed immediately to the gate. If forged, that failure diverts cleanly into Alternative Flow A1 (Detention Room). Using 'validates' asserts victory in the happy path and pushes failure into an explicit branch.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

'Validates' Kelimesinin Eklemeli (Additive) Mimari Gücü: Cockburn ve Martinelli'nin en radikal kuralı: 'Validates, never checks whether.' (Doğrular deyin, kontrol eder demeyin!). 'Checks whether' ifadesi iki ucu açık bir olasılıktır ve if-else şartlı dallanmalarını tetikler. Use case'in ana senaryosuna if-else sokulamaz! 'Sistem üyenin gecikmiş borcu olmadığını doğrular (BR-006)' dediğinizde, ana akışta bu adımın %100 başarılı olduğu mühürlenmiş olur. Bu tek bir kelime tercihi sisteme şu muhteşem mimari faydaları sağlar: 1) Eklemeli Yapı (Additive Structure): Ana senaryo baştan sona pürüzsüz akar; her bir başarısızlık ihtimali ise aşağıda bağımsız bir alternatif akış (A1, A2) olarak tanımlanır. 2) AI'a Hata Şablonu Verme: AI ajanı bu yapıyı gördüğünde kodun içinde spagetti if-else blokları kurmaz; doğrudan `if (!valid) throw new ValidationException()` veya Result/Either monadı üreterek akışı ilgili alternatif metoda delege eder.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

The Additive Architecture of 'Validates' vs 'Checks Whether': 'Checks whether' is an open-ended predicate that invites inline branching and nested conditionals within the main scenario. The main scenario must remain strictly linear and unconditionally successful. Writing 'System validates that password matches hash (BR-001)' asserts success in the primary flow. This single verb choice constructs the additive architecture of the specification: 1) The main scenario is an unbranching highway where every validation passes. 2) Deviations are strictly quarantined into numbered alternative flows. 3) For generative AI, this enforces clean architectural patterns: guard-clause rejections (`if (!valid) throw new DomainException()`) rather than tangled if-else pyramids.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: 'Validates' Kuralının Guard-Clause Mimarisiyle Temiz İcrası
package com.maang.aiup.validates;

public class ValidationGuardArchitecture {
    // ❌ CHECKS WHETHER (Spagetti if-else):
    // if (checkPassword()) { if (checkLimit()) { doSomething(); } else { error(); } }

    // ✅ VALIDATES (Guard Clauses - Şartnamedeki her 'validates' bir guard'dır):
    public static void processBorrowWithGuards(boolean isCopyAvailable, boolean isLimitOk) {
        // Step 4: System validates copy is available (A1)
        if (!isCopyAvailable) {
            handleAlternativeA1();
            return;
        }

        // Step 5: System validates loan limit (A2)
        if (!isLimitOk) {
            handleAlternativeA2();
            return;
        }

        // Ana akış dümdüz devam eder!
        System.out.println("Tüm doğrulamalar geçti, ana akış başarıyla bitti.");
    }

    private static void handleAlternativeA1() { System.out.println("Dallanma A1: Kopya yok."); }
    private static void handleAlternativeA2() { System.out.println("Dallanma A2: Limit aşıldı."); }

    void main() {
        processBorrowWithGuards(true, true);
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Şartnamede 'Sistem kullanıcının bakiyesini kontrol eder' yazıp bırakırsanız AI bakiye yetersiz olduğunda ne yapacağını bilemez ve kafasına göre kullanıcıya eksi bakiye tanımlayabilir! 'Sistem bakiyenin yeterli olduğunu doğrular (BR-002)' yazın.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Writing 'System checks customer balance' without asserting success leaves the failure path undefined. An AI agent might hallucinate overdraft credit. Write 'System validates that balance is sufficient (BR-002)' and bind failure to flow A1.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- 'Validates' ana senaryoda kesin başarıyı mühürler; 'checks whether' ise ucu açık if-else yaratır.
- Her 'validates' adımı başarısızlık ihtimalini isimlendirilmiş bir Alternatif Akışa devreder.
- Bu kural sayesinde ana akış dümdüz bir başarı otobanı gibi tertemiz kalır.

🎯 Key Architectural Takeaways

- 'Validates' asserts success in the main flow; 'checks whether' creates ambiguous branching.
- Every validation step delegates its failure outcome to a named alternative flow.
- This rule preserves the happy path as a linear, unbranching highway to the goal.

33

BÖLÜM: SENARYOLAR VE ALTERNATIF AKIŞLAR YAZMA SANATI | MODULE: WRITING SCENARIOS AND ALTERNATIVE FLOWS

Alternatif Akışları Keşfetme Sanatı: Cockburn'ün 3 Turlu Beyin Fırtınası Yöntemi

Finding Alternative Flows: Cockburn's 3-Pass Brainstorming Method

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
COCKBURN'ÜN 3 TURLU ALTERNATİF AKIŞ BULMA YÖNTEMİ

TUR 1: BEYİN FIRTINASI (BRAINSTORM - Asla Çözüm Yazma!)
- Senaryodaki her 'validates' adımını al: "Bu doğrulama patlarsa ne olur?"
- Aktör hataları: "Yanlış girdi, zaman aşımı (timeout), vazgeçme?"
- Dış sistem hataları: "Ödeme gateway yanıt vermezse, SMS servisi çökerse?"
- İç donanım arızaları: "Para sayma haznesi sıkışırsa?"

TUR 2: RASYONALİZE ET & BİRLEŞTİR (RATIONALIZE)
- Sistemin tespit edemeyeceği şeyleri sil! ("Üye şifresini unuttu" -> tespit edilemez!)
  Bunu "3 kere yanlış şifre girildi" olarak yeniden yaz!
- Aynı sonucu veren arızaları birleştir: "Kart çizik, okuyucu bozuk" -> "Kart Okunamadı."

TUR 3: ÇÖZÜMÜ YAZ (WRITE THE HANDLING)
- Hayatta kalan her duruma A1, A2, A3 diye temiz akış adımlarını yaz!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir dedektif gibi suç mahallini inceliyorsun: 1. Turda aklına gelen her çılgın şüpheyi tahtaya yazarsın: 'Hırsız bacadan mı girdi, helikopterle mi indi, içerideki biri mi yardım etti?' (Asla hemen karar verme!). 2. Turda saçma olanları elersin: 'Evde baca yok, helikopter sesi duyulmadı, demek ki içeriden biri yardım etti' (Sadeleştir!). 3. Turda ise hayatta kalan tek ihtimalin peşine düşüp tutuklama planını yazarsın. Üçünü aynı anda yapmaya çalışırsan beynin yanar!

🇬🇧 ENGLISH (ELI5)

Think of a forensic detective: in Pass 1, brainstorm every wild hypothesis without filtering: 'Did the intruder rappel from a blimp? Tunnel through the basement? Use a master key?' In Pass 2, rationalize: cross off the blimp because radar was clear; collapse lock scratches into 'Forced Entry'. In Pass 3, write the formal arrest warrant. Trying to invent the solution while brainstorming the failure conditions drains cognitive energy.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Cockburn'ün 3 Turlu Yöntemi ve Eksik Akışların Maliyeti: Yazılım sistemlerinin kod karmaşıklığının %80'i mutlu yolda değil; alternatif akışlarda ve hata yönetiminde yaşar. Bir insan projesinde unutulan bir hata akışı, test aşamasında bir mühendisin 'Peki ya kartta bakiye yoksa ne olacak?' sorusuyla ortaya çıkabilirdi. AI çağında ise unutulan her alternatif akış, yapay zekanın kendi kafasından uydurduğu sahte bir davranışa dönüşür! Bu yüzden beyin fırtınası opsiyonel bir lüks değil, zorunlu bir güvenlik adımıdır: 1) Tur 1: Çözümü düşünmeden sadece tüm başarısızlık ihtimallerini listele. 2) Tur 2: Sistemin algılayamayacağı insan niyetlerini sistemsel olaylara dönüştür. Örneğin 'Üye PIN kodunu unuttu' sistem tarafından bilinemez; sistem sadece 'PIN girme süresi aşıldı (Timeout)' durumunu algılayabilir. Birbirinin aynı olan 5 hata sebebini tek bir başlıkta topla ('Kaydetme Başarısız'). 3) Tur 3: Sadece bu süzgeçten geçen gerçek durumlara A1, A2 akışlarını yaz.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Cockburn's 3-Pass Brainstorming Protocol: Eighty percent of enterprise code complexity resides in failure topologies. In legacy shops, an unstated failure mode surfaced during manual testing when an observant engineer raised an edge-case question. With autonomous LLM coders, an omitted failure mode produces invented behavior. Brainstorming is therefore mandatory: 1) Pass 1 (Exhaustive Brainstorming): Enumerate every conceivable failure without designing remedies. Sweep through every 'validates' step, actor input errors, timeout events, and secondary system outages. 2) Pass 2 (Rationalization): Discard undetectable phenomena ('User forgot password' is undetectable; reframe as 'Max failed attempts exceeded'). Merge identical outcomes ('Magnetic stripe scratched', 'Chip corrupted' merge into 'Card Unreadable'). 3) Pass 3 (Handling): Author the formal steps and termination semantics for the surviving conditions.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: 3 Turlu Analizin Koddaki Karşılığı Olan Hata Hiyerarşisi
package com.maang.aiup.brainstorm;

public class AlternativeFlowHandling {
    // Tur 2'de sadeleştirilen ve sisteme mal edilen hata tipleri
    public sealed interface PaymentFailure permits CardUnreadable, InsufficientFunds, GatewayTimeout
{}

    public record CardUnreadable(String details) implements PaymentFailure {}
    public record InsufficientFunds(double required, double current) implements PaymentFailure {}
    public record GatewayTimeout(String gatewayName) implements PaymentFailure {}

    // Tur 3'te yazılan çözüm akışları (Pattern Matching ile kusursuz dallanma):
    public static void handleFailure(PaymentFailure failure) {
        switch (failure) {
            case CardUnreadable _ -> System.out.println("A1 Flow: Kart okunamadı, başka bir kart talep
edildi.");
            case InsufficientFunds f -> System.out.printf("A2 Flow: Bakiye yetersiz (Gereken: %.2f),
alternatif ödeme istendi.%n", f.required());
            case GatewayTimeout _ -> System.out.println("A3 Flow: Banka zaman aşımı, işlem kuyruğa
alındı.");
        }
    }

    void main() {
        handleFailure(new InsufficientFunds(100.0, 45.0));
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Alternatif akışları düşünürken hemen çözüm yazmaya kalkışmayın. Hatanın nasıl çözüleceğini düşünmek, yeni hata durumlarını keşfetme enerjinizi tüketir. Önce tüm arızaları listeleyin, sonra çözümleri yazın.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never attempt to write the handling logic while brainstorming failure conditions. Designing the solution exhausts the cognitive bandwidth required to uncover subtle system failure modes.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Alternatif akışlar 3 turda bulunur: 1. Beyin fırtınası yap, 2. Sadeleştir, 3. Çözümü yaz.
- Sistemin algılayamayacağı insan durumları ('şifreyi unuttu') sistemsel olaylara ('zaman aşımı') çevrilir.
- Aynı sonucu doğuran mikro hatalar tek bir genel akış başlığı altında birleştirilmelidir.

🎯 Key Architectural Takeaways

- Alternative flows are discovered in three passes: 1. Brainstorm, 2. Rationalize, 3. Author handling.
- Undetectable human states must be translated into detectable system events.
- Micro-failures yielding identical systemic outcomes must be rolled up into a unified flow.

34

BÖLÜM: SENARYOLAR VE ALTERNATIF AKIŞLAR YAZMA SANATI | MODULE: WRITING SCENARIOS AND ALTERNATIVE FLOWS

Bir Alternatif Akışın Anatomisi: Adıma Bağlı Tetikleyici & Açık Uçlu Akış Yasağı

Anatomy of an Alternative Flow: Anchored Triggers & Explicit Endings

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
┌──────────────────────────────────────────────────────────┐
│                BİR ALTERNATİF AKIŞIN KESİN ŞABLONU                │
└──────────────────────────────────────────────────────────┘

### A1: No Copies Available  ◄── [ Koşulun İsmi Başlık Olur ]

**Trigger:** No copy of the book is available (step 4) ◄── [ Hangi adımda koptu? ]
**Flow:**
1. System informs member that all copies are on loan.
2. System offers to place a reservation.
3. If member accepts, system records reservation (UC-005 Reserve Book).
4. Use case ends.  ◄── [ KESİN BİTİŞ! ASLA UCU AÇIK BIRAKILAMAZ! ]

🚫 YASAK: Bir akışın son satırında "Use case continues at step N." veya
         "Use case ends." yazmıyorsa, bu bir şartname hatasıdır (Spec Bug)!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir tren yolculuğundasın ve makas değiştirip yan raya girdin. Makasın başında tabela yazar: 'Makas 4. kilometrede ayrıldı (Tetikleyici)'. Yan rayda birkaç yüz metre gidersin. Rayın sonunda ise iki ihtimal vardır: Ya ray tekrar ana hatta bağlanır ('Yolculuk 5. kilometreden devam eder') ya da ray bir takozla son bulur ('Yolculuk burada biter!'). Eğer rayı boşlukta bırakırsan tren uçurumdan yuvarlanır! Alternatif akışların ucu asla açık bırakılamaz!

🇬🇧 ENGLISH (ELI5)

Riding a train onto a siding track: the switch is explicitly marked ('Diverted at Kilometer 4'). You travel down the spur line for 500 meters. At the end of the spur, only two geometric outcomes are physically possible: either the track merges back into the main line ('Use case continues at step 5'), or it strikes a hydraulic buffer stop ('Use case ends'). Leaving a track hanging in mid-air derails the train. Open-ended flows are specification bugs.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Alternatif Akış Anatomisi ve Açık Uçlu Akış Yasağı: Cockburn bir alternatif akışı 'minyatür bir use case' olarak adlandırır. Bir alternatif akışın 3 zorunlu bileşeni vardır: 1) İsimlendirilmiş Başlık: Koşulun adını taşır (`### A1: No Copies Available`). 2) Adıma Çapalanmış Tetikleyici (Anchored Trigger): Sistemin neyi tespit ettiğini ve ana senaryonun HANGİ adımından koptuğunu kesin olarak belirtir: `**Trigger:** No copy of the book is available (step 4)`. Bu çapa hem insan denetçinin dalı bulmasını sağlar hem de AI'ın hata yakalama kodunu ilgili metodun içine gömmesini garantiler. Eğer hata bir aralıkta çıkabiliyorsa 'at any time between steps 2 and 6' yazılır. 3) Açık Bitiş Kuralı (Explicit Ending): Bir alternatif akış tam olarak iki şekilde bitebilir: a) `Use case continues at step N.` (Düzeltme yapıldı, ana yola geri dönüldü). b) `Use case ends.` (Hedefe ulaşılamadı, işlem iptal edildi). Sonu belirtilmemiş açık uçlu bir akış şartname hatasıdır (Spec Bug); insan geliştirici bunu omuz silkerek geçiştirebilir ama yapay zekâ kendi kafasından uyduruk bir son seçer.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Anatomy of an Alternative Flow and the Prohibition of Open-Endedness: An alternative flow is a miniature use case nested within the specification. It mandates three non-negotiable structural components: 1) Condition-Based Header: Named precisely after the observed system condition (`### A2: Loan Limit Reached`). 2) Step-Anchored Trigger: Declares what the system detected and anchors explicitly to a numbered main-scenario step: `**Trigger:** Member already has maximum loans (step 5)`. The step anchor enables AI code generators to attach the exception interceptor to the precise functional invocation. 3) Deterministic Termination: Exactly two mechanical endings exist: `Use case continues at step N.` or `Use case ends.` An alternative flow lacking an explicit terminal statement is a severe specification bug. While a human engineer might infer an ending, an LLM will invent one arbitrarily.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Adıma Çapalanmış ve Kesin Bitişli Alternatif Akış Metodu
package com.maang.aiup.flows;

public class AlternativeFlowAnatomyDemo {
    public record ReservationResult(boolean completed, String nextStep) {}

    // A1: No Copies Available (step 4)
    public static ReservationResult handleAlternativeA1(String bookId, boolean userAcceptsReservation)
{
        System.out.println("Step 4'ten A1 akışına girildi: Kopya bulunamadı.");

        // Flow Step 1: System informs member
        System.out.println("A1.1: Üyeye tüm kopyaların ödünçte olduğu bildirildi.");

        // Flow Step 2 & 3: Offer & Record Reservation
        if (userAcceptsReservation) {
            System.out.println("A1.3: Rezervasyon kaydedildi (UC-005 Reserve Book).");
        }

        // Flow Step 4: KESİN BİTİŞ (Use case ends!)
        System.out.println("A1.4: Use case ends.");
        return new ReservationResult(true, "USE_CASE_ENDS");
    }

    void main() {
        handleAlternativeA1("BK-101", true);
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Bir alternatif akışın içine bir alternatif akış daha açıp (iç içe 3 katmanlı dallanma) şartnameyi labirente çevirmeyin! 2. seviyeden daha derin bir dallanma ihtiyacı doğduysa, o akış use case olmaktan çıkmış demektir; onu ayrı bir use case olarak sisteme ekleyin.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never nest alternative flows more than one level deep. If an alternative flow branches into secondary and tertiary sub-alternatives, the abstraction is broken: promote the flow into its own standalone sea-level use case.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Her alternatif akışın tetikleyicisi ana senaryodaki numaralı bir adıma çapalanır (step N).
- Bir akış yalnızca iki şekilde bitebilir: 'Use case continues at step N.' veya 'Use case ends.'
- Ucu açık bırakılmış akışlar birer şartname hatasıdır ve AI'ın uydurma kararlar almasına yol açar.

🎯 Key Architectural Takeaways

- Every alternative flow trigger must anchor to a numbered main step (step N).
- Flows terminate strictly via 'Use case continues at step N.' or 'Use case ends.'
- Open-ended flows are specification defects that provoke arbitrary AI hallucinations.

35

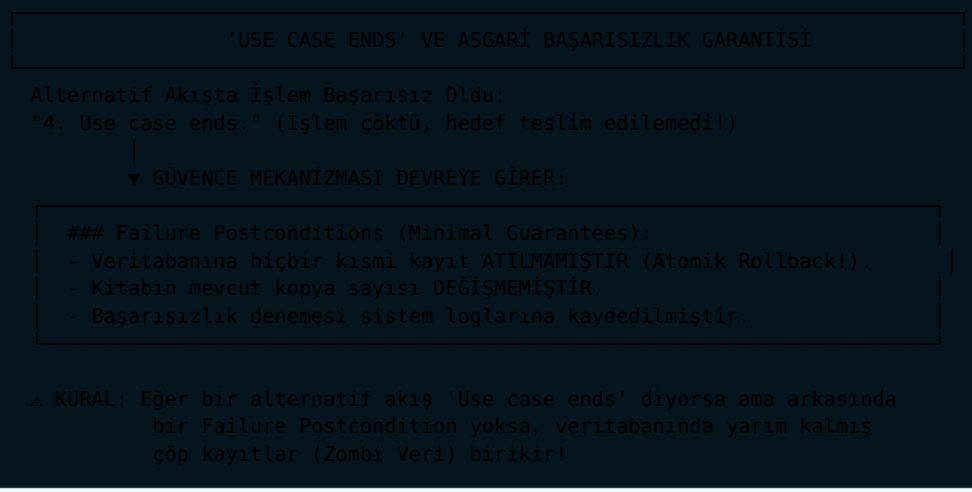
BÖLÜM: SENARYOLAR VE ALTERNATIF AKIŞLAR YAZMA SANATI | MODULE: WRITING SCENARIOS AND ALTERNATIVE FLOWS

Toplam Başarısızlığı Yönetmek: 'Use Case Ends' Neden Başarısızlık Sonkoşulunu Şart Koşar?

Handling Total Failures: Why 'Use Case Ends' Demands Failure Postconditions

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir ameliyat sırasında anestezi uzmanı 'Tansiyon çok düştü, ameliyatı hemen durduruyoruz (Use case ends!)' dediğinde doktorlar hastayı öylece masada bırakıp eve gitmez! Yarayı hemen dikerler, kanamayı durdururlar, hastayı yoğun bakıma alıp rapor yazarlar (Başarısızlık Sonkoşulu!). İşte 'Use case ends' dendiğinde sistemin arkasında yaralı hasta veya açık yara (yarım kaydedilmiş bozuk veri) bırakmaması bu garantiler sayesinde sağlanır.

🇬🇧 ENGLISH (ELI5)

If a surgical team encounters an emergency and aborts an operation ('Use case ends'), the surgeons do not drop their scalpels and stroll out of the hospital. They immediately suture the incision, stabilize vitals, and transfer the patient to intensive care (Failure Postconditions). 'Use case ends' is only tolerable because the minimal guarantees promise that the patient survives clean and uncorrupted.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

'Use Case Ends' ve İşlemsel Geri Alma (Rollback) Mimarisi: Bir alternatif akış 'Use case ends' dediğinde, birincil aktör hedefine ulaşamadan sistemden ayrılmış demektir. Cockburn bu durumun sistem açısından kabul edilebilir olmasının TEK BİR şartı olduğunu belirtir: 'Failure Postconditions' (Asgari Başarısızlık Garantileri) bölümünün eksiksiz tanımlanmış olması. Eğer bir akış başarısızlıkla sonuçlandığında arkasında silinmemiş bir geçici tablo kaydı, düşülmüş ama iade edilmemiş bir bakiye veya kilitli kalmış bir stok bırakıyorsa, sistem zehirlenir. AI açısından bu durum hayati bir mimari ayrıntıdır: LLM ajanı bir kod üretirken metottaki `try-catch` ve `\@Transactional` bloklarının içine ne yazacağını bu başarısızlık sonkoşullarına bakarak belirler. Eğer şartnamede 'Failure Postconditions: No partial data is stored' yazıyorsa, AI veritabanı işlemlerini atomik bir transaction ile sarar ve hata anında rollback tetikler.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Transactional Mechanics of Total Failure: Declaring 'Use case ends' signifies that the primary actor departs without achieving their goal. Cockburn asserts that this total failure is acceptable if and only if the system's Minimal Guarantees hold true. If an aborted flow leaves partial records, decremented inventory without a corresponding order, or orphaned connection handles, the system suffers state corruption. For AI code generators, the Failure Postconditions block acts as the specification for transactional exception handling. Stating 'No loan is recorded and book copy count is unchanged' instructs the AI to generate atomic `\@Transactional(rollbackFor = Exception.class)` boundaries with zero manual intervention.

⚡ Modern Java 25 & Clean Architecture Uygulaması ⚡ Java 25 & Clean Architecture Implementation

```
// Java 25: Failure Postconditions Sayesinde Otomatik Rollback Yapan İşlem Modeli
package com.maang.aiup.failure;

public class TransactionalFailureGuaranteeDemo {
    public static class InventoryRepository {
        private int stock = 10;
        public void decrement() { stock--; }
        public void rollback() { stock++; }
        public int getStock() { return stock; }
    }

    // Failure Postcondition Garantisi: Hata çıkarsa stok ASLA değişmez!
    public static void executeWithMinimalGuarantee(InventoryRepository repo, boolean forceFail) {
        int initialStock = repo.getStock();
        repo.decrement(); // Kısmi işlem yapıldı

        try {
            if (forceFail) {
                throw new RuntimeException("Ödeme başarısız oldu (Use case ends)!");
            }
        } catch (Exception e) {
            // Failure Postcondition gereği sistem durumu geri alır:
            repo.rollback();
            assert repo.getStock() == initialStock : "KRİTİK HATA: Stok tutarsız kaldı!";
            System.out.println("🛡️ Failure Postcondition doğrulandı: Stok eski haline döndürüldü -> "
+ repo.getStock());
        }
    }

    void main() {
        var repo = new InventoryRepository();
        executeWithMinimalGuarantee(repo, true);
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı ⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Bir alternatif akışın sonunda 'Use case ends' yazdıktan sonra sistemin ne halde kalacağını Failure Postconditions'a yazmayı unutursanız, AI hata anında hiçbir veriyi geri almayan ve veritabanını bozan bir kod üretecektir.

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Omitting Failure Postconditions while invoking 'Use case ends' guarantees that the generated code will lack transactional rollback handlers, resulting in corrupted database state under error conditions.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

• 'Use case ends' ancak ve ancak Başarısızlık Sonkoşulları (Minimal Guarantees) sağlandığında geçerlidir.

• Başarısızlık garantisi hiçbir kısmi verinin kaydedilmediğini ve sistemin temiz kaldığını taahhüt eder.

• Yapay zekâ veritabanı rollback ve loglama mekanizmasını bu sonkoşullara bakarak inşa eder.

🎯 Key Architectural Takeaways

• 'Use case ends' is valid only when backed by explicit Failure Postconditions.

• Failure guarantees promise that no partial state is persisted and pre-state is restored.

• AI synthesizes transactional rollback and audit handlers directly from failure postconditions.

36

BÖLÜM: SENARYOLAR VE ALTERNATIF AKIŞLAR YAZMA SANATI | MODULE: WRITING SCENARIOS AND ALTERNATIVE FLOWS

Tamamlanmış Şartnamenin Ritmi: Sıfır Açık Uç ve Kapalı Döngü (Closed Loop) Mimarisi

The Rhythm of a Finished Spec: Closed Loops with Zero Dangling Threads

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
┌──────────────────────────────────────────────────────────┐
│              KUSURSUZ ŞARTNAMENİN KAPALI DÖNGÜ (CLOSED LOOP) RİTMİ        │
└──────────────────────────────────────────────────────────┘

 1. HER ANA SENARYO ADIMI KESİNLİKLE BAŞARILIDIR. (Dallanma yok!)
    │
    ▼
 2. HER 'VALIDATES' ADIMININ TAM OLARAK BİR ALTERNATİF AKIŞ KARŞILIĞI VARDIR.
    (Step 4 Validates ──► A1 Flow)
    │
    ▼
 3. HER ALTERNATİF AKIŞ NUMARALI BİR ADIMDA BAŞLAR, KESİN BİR ÇIKIŞLA BİTER.
    (Trigger: Step 4 ──► Ends / Continues at Step N)
    │
    ▼
 4. HER 'ENDS' BİR BAŞARISIZLIK SONKOŞULUYLA (FAILURE POSTCONDITION) DESTEKLENİR.
    │
    ▼
 5. ATIFTA BULUNULAN HER (BR-XXX) İŞ KURALI LİSTESİNDE TANIMLIDIR.

 🔒 SIFIR AÇIK UÇ! Bu kapalı döngü yapay zekanın uydurma yapmasını İMKÂNSIZ kılar!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir elektrik devresini düşünün: Pilden çıkan kablo ampule gider, ampulden anahtara, anahtardan tekrar pile döner. Eğer devrede tek bir kablonun ucu havada sallanıyorsa ampul yanmaz (Açık devre!). Kusursuz bir use case şartnamesi kapalı bir elektrik devresi gibidir: Adım başlar, kurala bağlanır, hataya bağlanır, sonkoşula bağlanır ve döngü tamamlanır. Tek bir açık uç kalmadığında sistem ışık saçar!

🇬🇧 ENGLISH (ELI5)

Consider an electrical circuit: current flows from the battery to the switch, through the lamp, and returns cleanly to ground. If a single wire dangles disconnected, no current flows. A finished use case specification is a closed electrical circuit: every validation links to a rule, every rule links to a flow, and every flow terminates cleanly. When there are zero dangling threads, the AI generates software that works on the first run.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Kapalı Döngü Şartname Ritim ve Denetim Kuralları: Martinelli, kaliteli bir use case dökümanının 'kendi içinde kapalı bir döngü' (closed loop) oluşturması gerektiğini belirtir. Bu ritim review (inceleme) sırasında gözle veya `/spec-review` ile kolayca doğrulanabilir: 1) Kural 1: Ana senaryodaki her adım başarıyla biter; if-else veya dallanma barındırmaz. 2) Kural 2: Ana senaryoda kaç tane 'validates' kelimesi geçiyorsa, alternatif akışlarda tam olarak o sayıda A1, A2 akışı bulunmalıdır; ne eksik ne fazla! 3) Kural 3: Her alternatif akışın tetikleyicisi ana senaryodaki numaralı bir adıma çapalanır ve ya 'Use case continues at step N' ya da 'Use case ends' ile biter. 4) Kural 4: Senaryodaki her 'Use case ends' ifadesi, Failure Postconditions bölümündeki en az bir garantiyle örtüşür. 5) Kural 5: Metin içinde parantezle referans verilen her `(BR-XXX)` kuralı, dökümanın altındaki Business Rules listesinde eksiksiz mevcuttur. Bu 5 kural sağlandığında şartnamede tek bir 'açıkta kalmış ip' (dangling thread) kalmaz. İşte bu kapalı döngü ritmi, şartnameyi generatif (AI tarafından %100 kodlanabilir) kılan mucizevi sırdır.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

The Closed-Loop Specification Rhythm: A production-ready use case forms a closed relational loop where no logical thread dangles unmoored. Reviewers audit this rhythm via five invariants: 1) Invariant 1: Every main-scenario step succeeds unconditionally; no branches exist. 2) Invariant 2: Every step containing the verb 'validates' is matched by exactly one alternative flow resolving its failure. 3) Invariant 3: Every alternative flow anchors to a numbered step and terminates deterministically. 4) Invariant 4: Every 'Use case ends' terminal state is insured by a corresponding Failure Postcondition assertion. 5) Invariant 5: Every rule cited as `(BR-XXX)` exists in the Business Rules section, and every business rule is cited at least once. This closed loop provides total architectural containment: an AI agent cannot execute an unhandled path without directly violating a line of written text.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Kapalı Döngü (Closed Loop) Ritmine Sahip Kusursuz Use Case Denetçisi
package com.maang.aiup.closedloop;

public class ClosedLoopAuditor {
    public record SpecAuditMetrics(int validationSteps, int alternativeFlows, boolean
hasFailurePostconditions) {
        public boolean isClosedLoop() {
            // Ritim 2 & 4: Doğrulama sayısı akış sayısına eşit olmalı ve sonkoşul bulunmalıdır!
            return validationSteps == alternativeFlows && hasFailurePostconditions;
        }
    }

    void main() {
        // Borrow Book Şartnamesinin Ritim Sayımı: 3 Doğrulama (Adım 4,5,6) = 3 Akış (A1,A2,A3)
        var borrowBookSpec = new SpecAuditMetrics(3, 3, true);

        if (borrowBookSpec.isClosedLoop()) {
            System.out.println("✅ Şartname Kapalı Döngü (Closed Loop) ritmine sahip: Sıfır açık
uç!");
        } else {
            System.err.println("❌ Şartname ritmi bozuk: Eksik akış veya sonkoşul tespit edildi!");
        }
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Şartnamenizde bir 'System validates' adımı olup da altında o adımı karşılayan bir alternatif akış (A1, A2) yoksa bu ölümcül bir açıktır. AI o doğrulamayı kodlar ama başarısız olduğunda sistemin çökeceği bir RuntimeException fırlatıp kullanıcıyı ekranda dondurur.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Leaving a 'System validates' step without a corresponding Alternative Flow creates a critical flaw. The AI will generate a validation check that throws an unhandled server error, locking the frontend UI.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Kapalı döngü şartnamede her 'validates' bir akışa, her akış ise bir sonkoşula bağlanır.
- Hiçbir açık uç bırakılmadığında yapay zekanın kendi kafasından kural uydurması imkansızlaşır.
- Bu kapalı döngü ritmi hem insan incelemesini kolaylaştırır hem de otomatik test üretimini besler.

🎯 Key Architectural Takeaways

- A closed loop connects every validation to a flow, and every flow to a postcondition.
- Zero dangling threads makes it mathematically impossible for an AI to hallucinate logic.
- This closed-loop rhythm accelerates human reviews and powers automated test generation.